

Intelligent Document Extraction, Validation & API Platform

AI Engineer Internship – Technical Case Study

End-to-end financial document intelligence using OCR + Gemini + deterministic financial validation

Financial-services teams receive invoices and financial statements in native or scanned PDF / JPG / PNG formats.

Different layouts, image quality and OCR errors make manual extraction slow and inconsistent.

The required solution is a reusable Document Intelligence service that extracts complete visible information, validates key financial relationships, stores results and exposes them through a dashboard and REST APIs.

Target flow: Upload → Validate → Text Extraction/OCR → AI Extraction → Structured JSON → Financial Validation → Evidence → Persistence → PASS/FAILED → Dashboard + API.

Four supported document types

Invoice — visible fields, line items and tables.

Balance Sheet — headers, periods, currency and every visible financial line item.

Profit & Loss — income/expense line items, comparative periods and totals.

Cash Flow Statement — operating, investing, financing, FX/translation and cash reconciliation.

Input control

Document type is selected by the frontend and sent as request metadata.

Automated document schema classification is not required.

Extraction completeness and validation are the focus.

Common structured JSON schema is used across requests.

Accepted inputs

PDF / JPG / JPEG / PNG only.

Native PDFs and scanned/image-based documents.

Maximum 3 pages.

File type, empty/corrupt state, page count and basic integrity validated before extraction.

Implemented behavior

Unsupported formats are rejected.

Native PDF text is attempted first.

Scanned PDFs fall back to OCR.

Images are processed with Tesseract OCR.

Readable/support status and page count are included in the result.

- 4.1 Document Validation — validate before OCR/AI; graceful errors for unsupported, empty, corrupt or >3-page files.
- 4.2 Complete Field & Table Extraction — extract ALL meaningful visible information, including headers, dates, parties, totals, line items, tables and comparative values.
- 4.3 Evidence / Grounding — provide supporting source text where practical and page numbers where available; confidence is optional.
- 4.4 Validation — deterministic financial relationship checks using the actual extracted source fields.
- 4.5 Processing Status — PASS when required extraction and validations succeed; FAILED when the document cannot be processed.

Validation rules

Supported types: PDF, JPG/JPEG, PNG.

Readable document required.

Maximum 3 pages.

Empty/corrupt files fail before OCR or AI extraction.

Validation is input control, not document classification.

Structured result

file_type

is_supported

is_readable

page_count

status: PASS / controlled failure

Clear HTTP error response for invalid input.

Gemini receives the original document multimodally together with supporting OCR/native text context.

Prompt requires ALL meaningful visible fields, values, line items, tables and comparative periods.

Missing information is returned as null; unsupported inference is prohibited.

Current common Pydantic schema: `ExtractionResponse` → `fields`, `line_items`, `tables`.

Tables use structured rows and period/value objects; invoice line items preserve quantity, unit price, amount, tax rate and additional fields.

What is returned

Each extracted field can include evidence text.

Page number is included where available.

Invoice line items and table rows also carry evidence/page context.

Confidence scoring is optional and is not implemented in the current common schema.

Why it matters

Allows important extracted values to be traced to the source.

Reduces ambiguity during evaluation.

Supports debugging of OCR/AI extraction.

Preserves source context without asking the model to invent missing data.

Tolerance: Decimal arithmetic with 0.01 tolerance.

Invoice: $\text{quantity} \times \text{unit price} \approx \text{line total}$; $\text{line totals} \approx \text{subtotal}$; $\text{subtotal} + \text{tax} - \text{discount} \approx \text{total}$; tax-rate check when a single consistent rate is available.

Balance Sheet: $\text{total assets} \approx \text{capital} + \text{reserves/surplus} + \text{minority interest} + \text{deposits} + \text{borrowings} + \text{other liabilities/provisions}$; checked independently for each period.

P&L: income, expenditure, profit-before-minority and attributable-profit reconciliations; comparative periods are checked independently.

Cash Flow: $\text{operating} + \text{investing} + \text{financing} + \text{FX/translation} + \text{applicable adjustments} \approx \text{net increase}$; $\text{opening cash} + \text{net increase} + \text{adjustments} \approx \text{closing cash}$; parentheses are treated as negative.

Unavailable required inputs produce NOT_APPLICABLE rather than invented values.

PASS

Required fields are extracted accurately.

Required financial validations pass.

Validation checks are returned in structured JSON.

Processed result is persisted and appears in the dashboard.

FAILED / controlled errors

Invalid, corrupted or unsupported document.

Unable to process the document.

Downstream model/API failure.

Structured error response is returned without exposing stack traces or secrets.

POST	/api/v1/documents/process	Upload + process PDF/JPG/PNG
GET	/api/v1/documents/{document_name}	Retrieve latest stored result
GET	/api/v1/documents	List processed documents for dashboard
GET	/api/v1/health	Service health check
GET	/docs	Swagger / OpenAPI documentation

POST request: multipart/form-data
file = uploaded document
document_type = invoice | balance_sheet | profit_and_loss | cash_flow_statement

Processing is synchronous for this assignment and the result is persisted.

Successful response contains

document_name + supplied document_type

processing_status

file_validation metadata

extracted_data: fields / line_items / tables

validation: checks / overall_status / issues

processing_metadata: OCR usage, multimodal extraction, timestamps and processing time

Controlled errors

HTTP status codes reflect the failure.

Consistent JSON error structure.

Example: GEMINI_QUOTA_EXCEEDED when the configured Gemini API quota is exhausted.

Unsupported/invalid inputs fail gracefully.

No stack traces or secrets exposed to users.

Implemented UI

HTML / CSS / Vanilla JavaScript frontend.

Document type selection + PDF/JPG/PNG upload + Process action.

API connection status.

Dashboard/history with document name, type, status, processed time and page count.

Result view with extracted fields, invoice line items, tables and financial validation.

Raw JSON view and visual highlighting for missing values / failed validations.

Integration

Frontend and API are served by the same FastAPI application.

Dashboard is backed by persistent stored results.

Frontend calls the deployed backend/API.

Confidence is not shown because confidence scoring is optional and not part of the current schema.

Live deployment

Frontend: <https://document-intelligence-x363.onrender.com/>

Backend/API: <https://document-intelligence-x363.onrender.com>

Swagger: <https://document-intelligence-x363.onrender.com/docs>

Health: <https://document-intelligence-x363.onrender.com/api/v1/health>

Public GitHub: <https://github.com/c639-afk/document-intelligence>

Deployment design

Dockerized application.

Render free-tier service.

Same deployed FastAPI service serves frontend + REST API.

Neon PostgreSQL used for deployment persistence.

No API keys or credentials committed to GitHub.

Gemini API is used through its free tier.

```
document-intelligence/
├── backend/
│   ├── app/
│   │   ├── api/routes/
│   │   ├── core/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   └── repositories/
│   ├── tests/
│   └── requirements.txt
├── frontend/
├── docs/
│   ├── architecture.png
│   ├── AI_USAGE.md
│   └── solution_presentation.pdf
├── sample_outputs/
│   ├── invoice_sample.json
│   ├── balance_sheet_sample.json
│   ├── profit_and_loss_sample.json
│   └── cash_flow_sample.json
├── Dockerfile
├── .env.example
└── .gitignore
```

Engineering choices

Modular separation of validation, OCR/parsing, AI extraction, financial validation, persistence and API.

Pydantic structured outputs enforce a fixed common schema.

SQLAlchemy abstracts persistence.

Decimal arithmetic provides deterministic financial calculations.

Environment variables hold configuration and secrets.

Reliability & security

Controlled exceptions for invalid files and downstream failures.

Upload validation before processing.

No hardcoded document outputs or expected validation answers.

Structured error responses.

Processing metadata retained for troubleshooting.

Production hardening includes stronger observability, rate limiting and operational controls.

Reliability — malformed/unsupported documents and downstream failures return controlled errors rather than crashing the service.

Security — no embedded secrets; uploads are validated and environment variables are used for credentials.

Maintainability — responsibilities are separated into services, repositories, schemas, routes and models.

Observability — processing metadata, OCR/model status and structured errors support troubleshooting; logging hardening remains a final cleanup item.

Usability — dashboard makes processed documents, extracted values, tables and validation results easy to inspect.

Performance — synchronous processing is acceptable for the 3-page assessment scope; processing time is recorded.

Persistence — processed metadata and complete result JSON are stored and surfaced through the dashboard.

Automated tests

File validation tests.

Financial validation tests.

API flow tests.

Current local suite: 24 tests passing.

SQLite is used for local test isolation.

Required test scenarios

All four supported document types.

At least one scanned/image-based document.

Financial validation failure or missing/unreadable value.

Unsupported/invalid file.

Sample JSON outputs with validation calculations.

Live deployed verification remains dependent on Gemini quota availability.

Public GitHub repository — complete source code.

Live frontend URL and live backend/API URL.

Swagger/OpenAPI documentation and health endpoint.

README with setup, architecture, API, persistence, deployment, limitations and production notes.

Architecture diagram.

Sample JSON outputs for all four document types.

AI/tool usage declaration.

Solution Presentation PDF.

Known limitations and production-improvement notes.

Short demo / walkthrough during the interview.

Documented

Technology stack and reasons for major choices.
Local setup and environment variables.
Live deployment URLs.
API request examples.
OCR/document parsing and Gemini model/free-tier usage.
Financial validation rules and tolerance.
Database/persistence approach.
Known limitations.

Also covered

Production deployment improvements.
AI coding assistant usage.
Architecture overview.
Project structure.
Testing instructions.
Security / secrets handling.
Error handling and API behavior.
Submission checklist.

AI & document processing

Google Gemini API — multimodal financial document extraction.

Gemini API is used through the free tier.

Tesseract — local OCR for images/scanned PDFs.

pypdf — native PDF text extraction.

pdf2image + Pillow — PDF page/image handling.

AI-assisted development

ChatGPT was used for requirement review, architecture planning, debugging assistance, validation review and documentation.

AI assistance was declared in docs/AI_USAGE.md.

Financial validation is deterministic and independent of the LLM.

Missing required validation inputs are not invented; they become NOT_APPLICABLE.

High-weight areas addressed

End-to-end deployed solution.

Extraction across four financial document types.

REST API + Swagger.

Frontend dashboard + database persistence.

Deterministic financial validation.

Testing + public GitHub + documentation.

Final submission actions

Run final live extraction tests after Gemini free-tier quota reset.

Verify all four supplied documents through the deployed API.

Verify scanned/image processing and failure scenarios.

Complete final logging hardening if required.

Push final presentation and documentation changes to public GitHub.

The platform converts heterogeneous financial documents into consistent structured JSON.

OCR/native extraction handles both digital and scanned inputs.

Gemini performs multimodal field and table extraction while deterministic code performs financial validation.

Results are persisted in PostgreSQL and exposed through REST APIs and a dashboard.

The design follows the case-study flow and keeps extraction, validation, persistence and API concerns separated.

Primary remaining step before final submission: complete live verification across the four documents after the Gemini free-tier quota resets.